Lab Report: Gamma Radiation

Chhaya Ramnani (UID: u8336188)

Lab partners: Tsz Wing Candice Ng, Chinatsu Komada

2026-10-21

Imports and Functions

All fits are straight lines by orthogonal distance regression, so the uncertainty on both axes is included. Counts are Poisson, so the uncertainty on a count N is $\sqrt{N}$.

```
from dataclasses import dataclass
from math import log
import matplotlib.pyplot as plt
import numpy as np
from scipy import odr
from uncertainties import ufloat
from uncertainties import unumpy as unp


@dataclass
class odr_fit_result:
    parameters: np.ndarray
    parameter_errors: np.ndarray
    r_squared: float
    x_nom: np.ndarray
    y_nom: np.ndarray
    x_err: np.ndarray
    y_err: np.ndarray


def linear_model(p, x):
    return p[0] * x + p[1]


def odr_fit(x, y, model_func, beta0):
    x_nom, x_err = np.array([v.n for v in x]), np.array([v.s for v in x])
    y_nom, y_err = np.array([v.n for v in y]), np.array([v.s for v in y])
```

```
    data = odr.RealData(x_nom, y_nom, sx=x_err, sy=y_err)
    odr_result = odr.ODR(data, odr.Model(model_func), beta0=beta0).run()

    residuals = y_nom - model_func(odr_result.beta, x_nom)
    r_squared = 1 - np.sum(residuals**2) / np.sum((y_nom - np.mean(y_nom)) ** 2)

   return odr_fit_result(odr_result.beta, odr_result.sd_beta, r_squared, x_nom,
y_nom, x_err, y_err)


def sigma_test(k1, sk1, k2, sk2, label="parameter"):
    sigma_diff = abs(k1 - k2) / np.sqrt(sk1**2 + sk2**2)
    print(f"\nSIGMA TEST ({label})")
    print(f"value 1 : {k1:.3f} +/- {sk1:.3f}")
    print(f"value 2 : {k2:.3f} +/- {sk2:.3f}")
    print(f"discrepancy: {sigma_diff:.2f} sigma -> {'AGREE' if sigma_diff <= 3
else 'DISAGREE'}")
    return sigma_diff


def plot_fit(result, model_func, xlabel, ylabel, title, filename):
    x_smooth = np.linspace(min(result.x_nom), max(result.x_nom), 200)
    residuals = result.y_nom - model_func(result.parameters, result.x_nom)

       fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(7, 7), sharex=True,
gridspec_kw={"height_ratios": [3, 1]})
              ax1.errorbar(result.x_nom,   result.y_nom,   xerr=result.x_err,
yerr=result.y_err, fmt="o", capsize=3, label="Data")
         ax1.plot(x_smooth,  model_func(result.parameters,  x_smooth),  "k--",
label=f"ODR fit ($R^2$={result.r_squared:.4f})")
    ax1.set_ylabel(ylabel)
    ax1.set_title(title)
    ax1.grid(True, alpha=0.6)
    ax1.legend()

   ax2.errorbar(result.x_nom, residuals, yerr=result.y_err, fmt="o", capsize=3,
color="crimson")
    ax2.axhline(0, color="black", linestyle="--")
    ax2.set_xlabel(xlabel)
    ax2.set_ylabel("Residual")
    ax2.grid(True, alpha=0.6)
```

```
    plt.tight_layout()
    plt.savefig(f"graphs/{filename}.png", dpi=300)
    plt.show()


# counts N in time t are Poisson, so the uncertainty on N is sqrt(N)
def rate(N, t):
    return ufloat(N, np.sqrt(N)) / t
```

Pre-Lab Questions

Question 1

Gamma rays spread out from a small source in all directions, so at distance r the same power is spread over a sphere of area $4\pi r^2$. So

$$I \propto \frac{1}{r^2}$$

Question 2

The linear attenuation coefficient of iron is

$$\mu = \sigma\rho = (0.1\ \mathrm{cm}^2/\mathrm{g})(7.874\ \mathrm{g/cm}^3) = 0.787\ \mathrm{cm}^{-1}$$

Blocking 75% means $I/I_0 = 0.25$, so from $I = I_0 e^{-\mu x}$:

$$x = \frac{\ln 4}{\mu} = \frac{1.386}{0.787\ \mathrm{cm}^{-1}}$$

$$\boxed{x \approx 1.76\ \mathrm{cm}}$$

Aim

To learn how to operate a Geiger-Müller counter to detect gamma radiation, and to examine how radiation intensity changes with distance from a source and how well a material absorbs gamma radiation.

Theory

The intensity of the radiation after a thickness x of a material is

$$I(x) = I_0 e^{-\mu x}$$

where μ is the linear absorption coefficient of the material. Taking the negative natural log of the ratio gives

$$-\ln\left(\frac{I(x)}{I_0}\right) = \mu x$$

so the gradient of $-\ln(I(x)/I_0)$ against x is μ. The mass attenuation coefficient is $\sigma = \mu/\rho$, where ρ is the density.

The intensity is proportional to $1/d^2$ (the inverse square law), so $\ln R = -2\ln r + \text{const}$, and $\ln R$ against $\ln r$ should have a gradient of -2.

Uncertainty propagation. The uncertainty on a count N is $\sqrt{N}$, so the rate $R = N/t$ has $\sigma_R = \sqrt{N}/t$. Subtracting the background gives

$$\Delta x_{\text{true}} = \sqrt{\Delta x_{\text{measured}}^2 + \Delta x_{\text{background}}^2}$$

For a log, $\sigma_{\ln x} = \sigma_x/x$, and for the attenuation graph

$$\Delta\left[-\ln\left(\frac{I(x)}{I_0}\right)\right] = \sqrt{\left(\frac{\Delta I(x)}{I(x)}\right)^2 + \left(\frac{\Delta I_0}{I_0}\right)^2}$$

Two values are compared as $|k_1 - k_2|/\sqrt{\sigma_1^2 + \sigma_2^2}$, and agree if this is at most 3σ.

Procedure

First, we plugged the Geiger-Müller counter into the computer and set the Vernier software to record 1 sample per second for 120 s. The counter was held with a clamp on a retort stand, above a weak Barium-133 sample in a lead housing. Every reading was 120 s long, and we used the mean count per second.

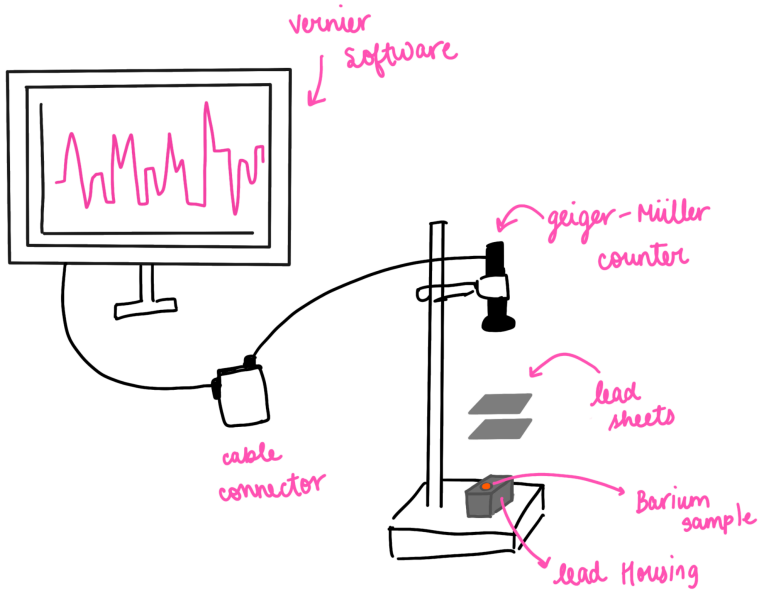


Figure 1: Experimental setup, peep the pink accents hehe

We recorded the background first, getting a mean of 0.358 counts/s, which is $N_{\text{bg}} = 43$ counts in 120 s. All the means below were converted to counts by multiplying by 120.

For the distance, we moved the clamp from 3.5 cm to 12.5 cm from the source in 1 cm steps, reading the ruler straight on to avoid parallax, and took a reading at each. We stopped at 12.5 cm because the count was close to background.

For the attenuation, we kept the counter at a fixed height of 6.5 cm and took the reading with no shield, 1.875 counts/s, as I_0. We measured the thickness of one lead sheet with vernier calipers, 0.22 ± 0.01 cm, and took readings with 1, 2, 3 and 4 sheets between the sample and the counter.

Results

Distance dependence

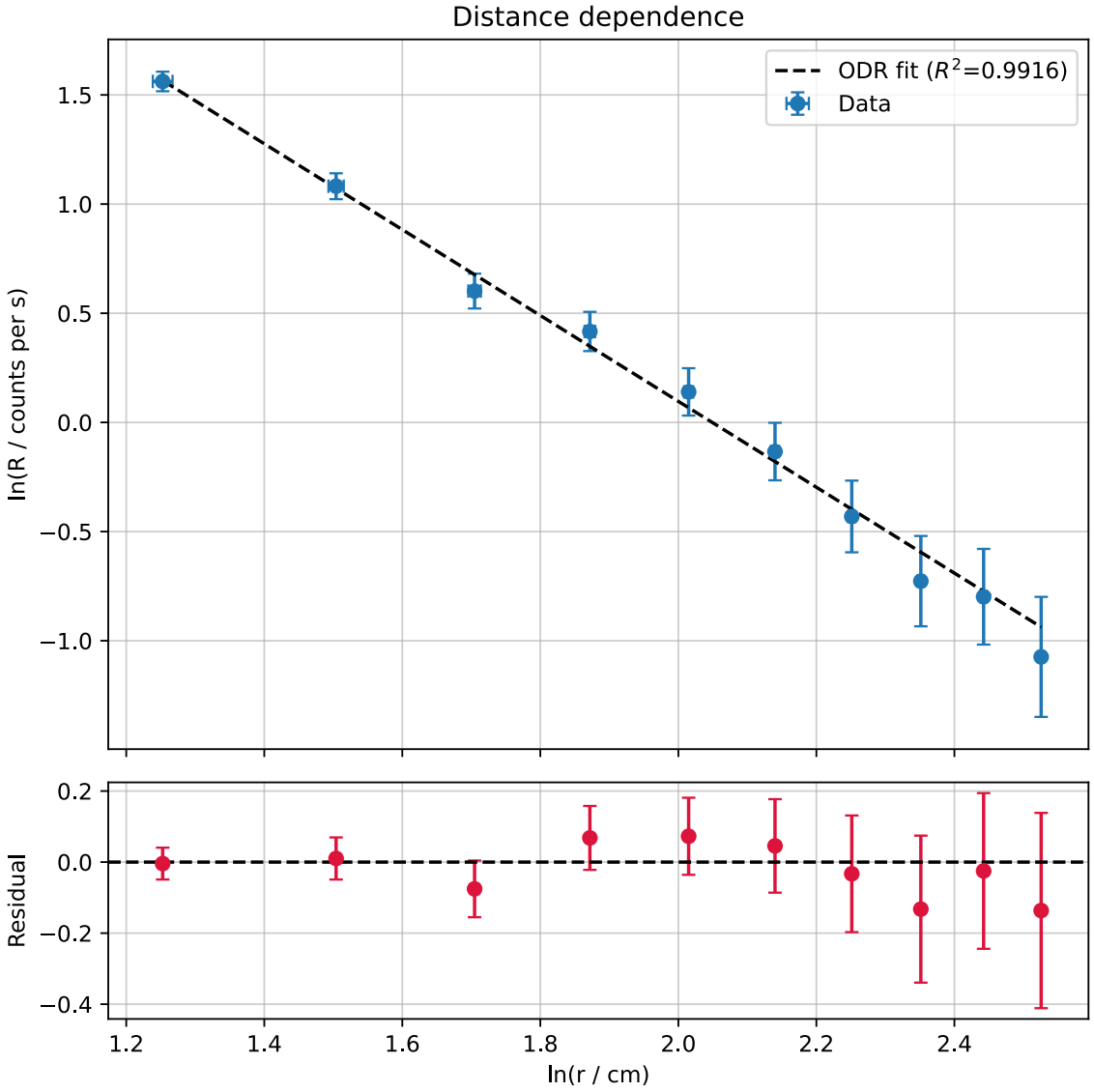
```
t_bg, N_bg = 120, 43          # s, counts: background (mean 0.358 counts/s over
120 s)
t = 120                       # s, counting time at each distance (1 sample/s)
sr = 0.05                     # cm, uncertainty
# distance (cm), counts
distance_data = [(3.5, 615), (4.5, 397), (5.5, 262), (6.5, 225), (7.5, 181),
(8.5, 148), (9.5, 121), (10.5, 101), (11.5, 97), (12.5, 84)]

if distance_data:
    bg = rate(N_bg, t_bg)
    r = [ufloat(d, sr) for d, _ in distance_data]
    R = [rate(N, t) - bg for _, N in distance_data]

    res = odr_fit(unp.log(r), unp.log(R), linear_model, beta0=[-2, 0])
    slope = ufloat(res.parameters[0], res.parameter_errors[0])
    print(f"\nln R vs ln r: slope = {slope:.3f}, R^2 = {res.r_squared:.4f}")
    sigma_test(slope.n, slope.s, -2, 0, label="slope vs inverse square")
    plot_fit(res, linear_model, "ln(r / cm)", "ln(R / counts per s)", "Distance
dependence", "distance")
```

```
ln R vs ln r: slope = -1.967+/-0.049, R^2 = 0.9916

SIGMA TEST (slope vs inverse square)
value 1 : -1.967 +/- 0.049
value 2 : -2.000 +/- 0.000
discrepancy: 0.68 sigma -> AGREE
```



Distance ± 0.05 (cm)	Counts N	Mean (counts/s)	Std dev	R_{measured} (counts/s)	R, background corrected (counts/s)
3.5	615	5.125	2.325	5.125 ± 0.207	4.767 ± 0.214
4.5	397	3.308	1.918	3.308 ± 0.166	2.950 ± 0.175
5.5	262	2.183	1.455	2.183 ± 0.135	1.825 ± 0.146

Distance $\pm$ 0.05 (cm)	Counts N	Mean (counts/s)	Std dev	R_{measured} (counts/s)	R, background corrected (counts/s)
6.5	225	1.875	1.464	1.875 ± 0.125	1.517 ± 0.136
7.5	181	1.508	1.257	1.508 ± 0.112	1.150 ± 0.125
8.5	148	1.233	1.075	1.233 ± 0.101	0.875 ± 0.115
9.5	121	1.008	1.025	1.008 ± 0.092	0.650 ± 0.107
10.5	101	0.842	0.926	0.842 ± 0.084	0.483 ± 0.100
11.5	97	0.808	0.843	0.808 ± 0.082	0.450 ± 0.099
12.5	84	0.700	0.816	0.700 ± 0.076	0.342 ± 0.094

The background is 0.358 ± 0.055 counts/s. The gradient of $\ln R$ against $\ln r$ is -1.967 ± 0.049 ($R^2 = 0.992$), which is 0.68σ from the -2 expected, so the inverse square law holds.

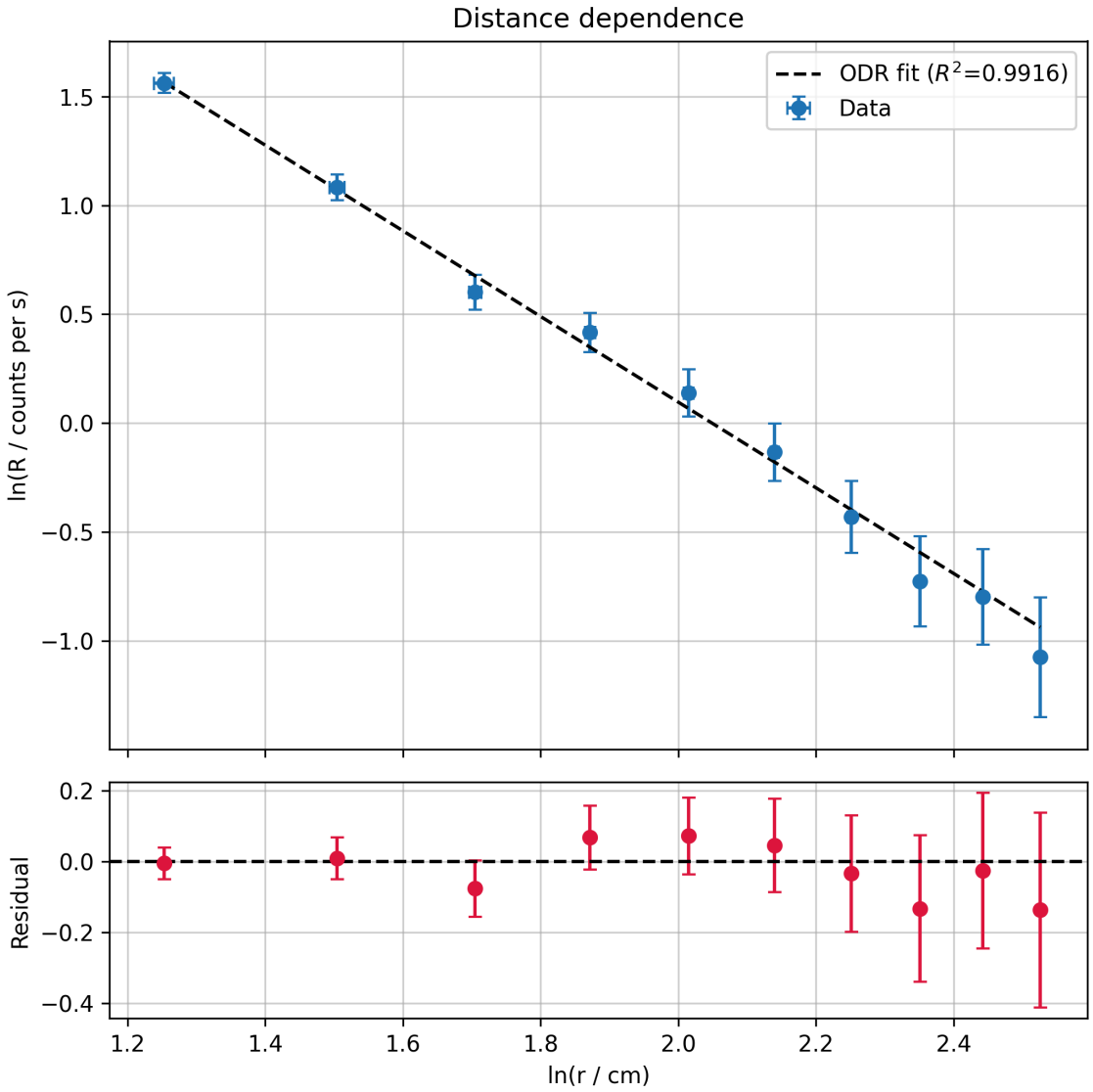


Figure 2: Distance dependence: fit and residuals

Attenuation by lead

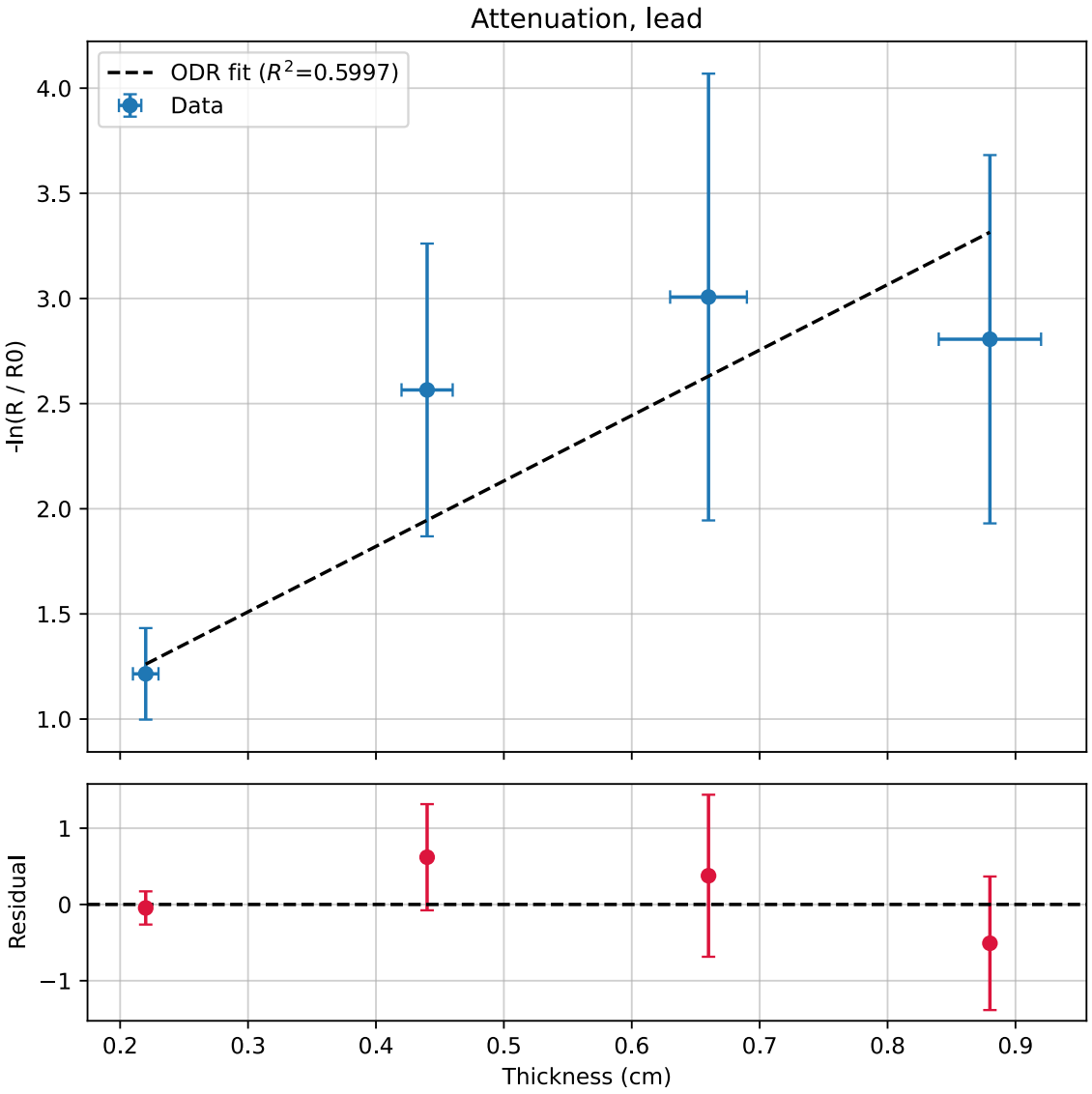
```
# sheets: lead sheet readings with 1, 2, 3, 4 sheets between the sample and
the counter
t_att = 120                    # s, counting time at each thickness
t_sheet, sx = 0.22, 0.01       # cm, thickness of one sheet and its uncertainty
(vernier caliper)
N0 = 225                       # counts with no shield (1.875 counts/s)
N_sheets = [97, 57, 52, 54]    # counts with 1, 2, 3, 4 sheets
sigma_nist, rho = 0.3074, 11.35   # NIST lead at 0.356 MeV (interpolated), cm^2/
```

```
g and g/cm^3

if N_sheets:
    bg = rate(N_bg, t_bg)
    R0 = rate(N0, t_att) - bg
    x = [ufloat(n * t_sheet, n * sx) for n in range(1, len(N_sheets) + 1)]
    R = [rate(n, t_att) - bg for n in N_sheets]

    res = odr_fit(x, [-unp.log(r / R0) for r in R], linear_model, beta0=[1, 0])
    mu = ufloat(res.parameters[0], res.parameter_errors[0])
    print(f"\nlead: mu = {mu:.3f} /cm, R^2 = {res.r_squared:.4f}")
    print(f"sigma = mu / rho = {mu / rho:.3f} cm^2/g")
    plot_fit(res, linear_model, "Thickness (cm)", "-ln(R / R0)", "Attenuation,
lead", "attenuation_lead")
    sigma_test(mu.n, mu.s, sigma_nist * rho, 0, label="lead mu vs NIST, /cm")
```

```
lead: mu = 3.112+/-0.951 /cm, R^2 = 0.5997
sigma = mu / rho = 0.274+/-0.084 cm^2/g
```




```
SIGMA TEST (lead mu vs NIST, /cm)
value 1 : 3.112 +/- 0.951
value 2 : 3.489 +/- 0.000
discrepancy: 0.40 sigma -> AGREE
```

Sheets	Thickness (cm)	Counts N	Mean (counts/s)	R, background corrected (counts/s)	$-\ln(R/R_0)$
0	0	225	1.875	1.517 ± 0.136	0
1	0.22 ± 0.01	97	0.808	0.450 ± 0.099	1.22 ± 0.22
2	0.44 ± 0.02	57	0.475	0.117 ± 0.083	2.56 ± 0.70
3	0.66 ± 0.03	52	0.433	0.075 ± 0.081	3.01 ± 1.06
4	0.88 ± 0.04	54	0.450	0.092 ± 0.082	2.81 ± 0.88

The gradient gives $\mu = 3.11 \pm 0.95$ cm^{-1}, and with $\rho = 11.35$ g/cm^3 for lead, $\sigma = \mu/\rho = 0.274 \pm 0.084$ cm^2/g.

Via NIST, σ_{ref} for lead is 0.4031 cm^2/g at 0.3 MeV and 0.2323 cm^2/g at 0.4 MeV. The Barium-133 energy peak is at 0.356 MeV, so we used linear interpolation, giving $\sigma_{\text{ref}} = 0.3074$ cm^2/g ($\mu_{\text{ref}} = 3.49$ cm^{-1}). Our μ is 0.40σ from this, so the two agree.

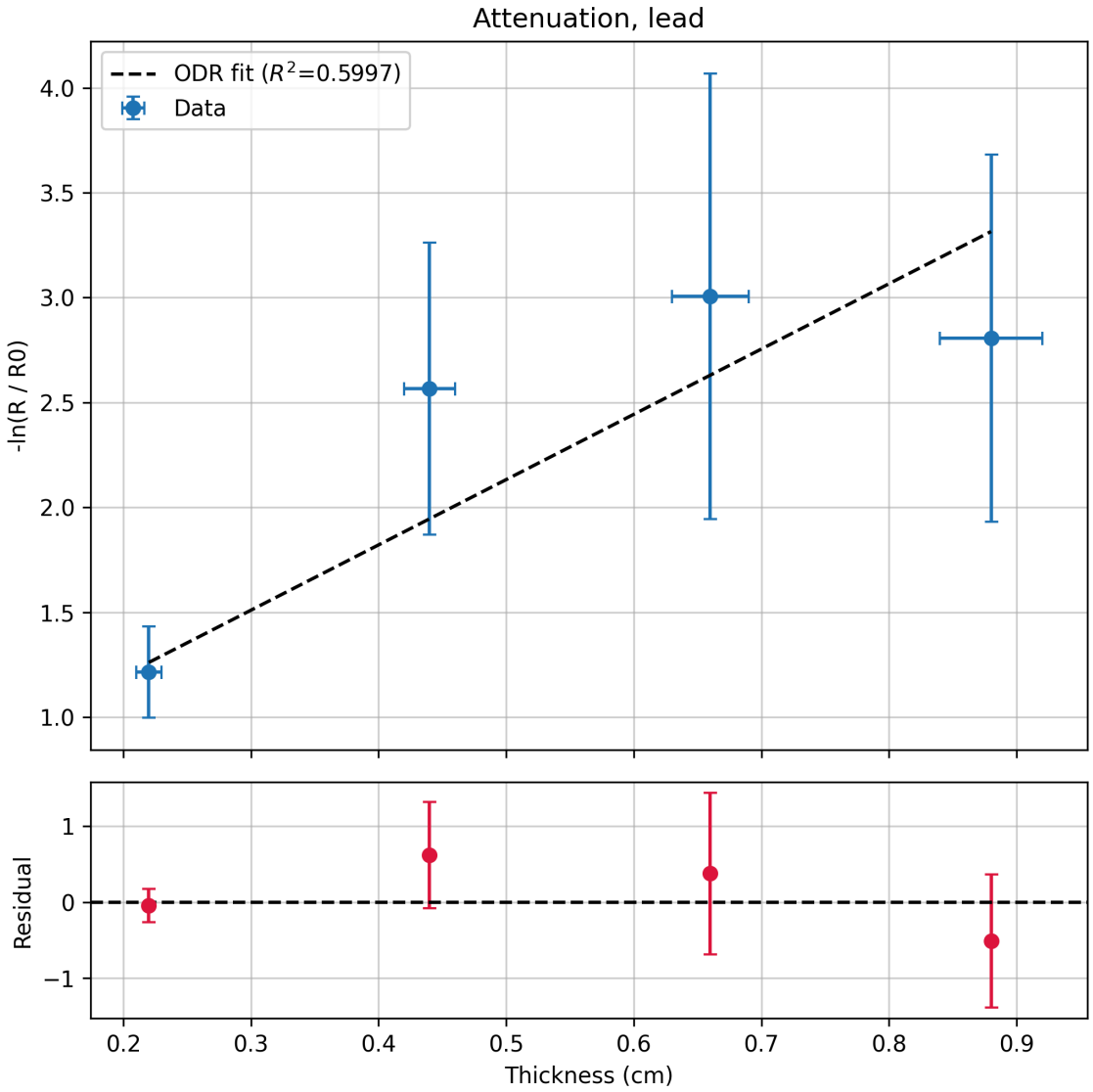


Figure 3: Attenuation by lead: fit and residuals

Limitations & Possible Sources of Discrepancy

Several steps were taken to reduce the uncertainties. The distance was measured from the source to the counter rather than from the housing, the ruler was read straight on to avoid parallax, and every reading was 120 s long with the background subtracted. The counter stayed at the same height for every sheet, and both graphs used orthogonal distance regression so the uncertainty in distance and thickness is included.

1. Weak source: After the background is subtracted the counts behind 2 to 4 sheets are very small, so the uncertainties are large (± 0.95 cm^{-1} on μ) and the fit is not perfect ($R^2 = 0.60$). The readings should be repeated and averaged to reduce this.

2. Beta and alpha radiation: The counter detects beta and alpha radiation as well as gamma. The first sheet probably blocks these, so it removes far more than the others which affects the gradient.

3. Density: The density of lead was taken from NIST and treated as exact.

Conclusion

The count rate follows the inverse square law, with a gradient of -1.967 ± 0.049 against the expected -2 (!!). The mass attenuation coefficient of lead for the Barium-133 gamma rays is 0.274 ± 0.084 cm^2/g, which agrees with the NIST value of 0.3074 cm^2/g within 3σ (0.40σ), though with a large uncertainty.